

Les P-uplets



Cours 7 : p-uplets

Un **p-uplet** (ou **n-uplet**) est un type construit permettant de regrouper des valeurs de différents types et d'y accéder grâce à un indice (**indexé**), tout comme une **liste**. Cependant, contrairement aux listes, il n'est pas possible d'en modifier le contenu a posteriori (**immuable**).

Pour créer un p-uplet, il suffit de séparer les différentes valeurs par des virgules. Cependant, pour plus de clarté, il est préférable de rajouter des parenthèses autour des différents éléments.

```
# création de mon p-uplet des élèves de seconde et leur classe
eleves_seconde = ("Laura", 204, "Jolan", 201, "Johan", 204)

# création d'un p-uplet vide
eleves_redoublants = ()

# création d'un p-uplet avec 1 seul élément
eleves_erasmus = ("Laura",)
```

Accès aux éléments et taille d'un p-uplet

Il est possible d'accéder aux éléments d'un p-uplet non vide à l'aide de leur index, comme pour les **listes** et les **str**. On utilise les crochets pour indiquer l'indice de la position, en commençant à 0.

```
elements_chimiques = ('H', 'Li', 'Na', 'K')
helium = elements_chimiques[0]
potassium = elements_chimiques[3] # similaire à elements_chimiques[-1]
```

Pour connaître la taille d'un p-uplet (c'est-à-dire le nombre d'éléments présents), on peut utiliser la fonction Python **len()**, exactement comme pour une liste.

```
elements_chimiques = ('H', 'Li', 'Na', 'K')
print(len(elements_chimiques))
>> 4
```

Il est également possible d'accéder à plusieurs éléments en même temps. Cette méthode, appelée **slicing**, fonctionne de la même manière que pour les listes ou les chaînes de caractères (**str**).

```
elements_chimiques = ('H', 'Li', 'Na', 'K')

# afficher le deuxième et troisième élément
print(elements_chimiques[1:3])
>> ('Li', 'Na')

# afficher tous les éléments à partir du troisième
print(elements_chimiques[2:])
>> ('Na', 'K')

# afficher tous les éléments jusqu'au troisième
print(elements_chimiques[:3])
>> ('H', 'Li', 'Na')
```

Opérations sur les p-uplets

Comme pour les chaînes de caractères (`str`) et les listes, il est possible d'utiliser les opérateurs `+` et `*` sur les p-uplets. L'opérateur `+` permet de **concaténer** deux p-uplets, c'est-à-dire de créer un nouveau p-uplet contenant les deux p-uplets de l'opération.

```
elements_chimiques = ('H', 'Li', 'Na', 'K')
elements_manquants = ('Fe', 'Be', 'Ca')
print(elements_chimiques + elements_manquants)
>> ('H', 'Li', 'Na', 'K', 'Fe', 'Be', 'Ca')
```

L'opérateur `*` permet de multiplier un p-uplet avec un entier `n`, créant un nouveau p-uplet contenant `n` copies du p-uplet original, comme pour les listes.

```
saisons = ("été", "printemps", "automne", "hiver")
print(saisons*2)
>> ("été", "printemps", "automne", "hiver", "été", "printemps", "automne", "hiver")
```

Tester l'appartenance

Pour vérifier si un élément est présent dans un p-uplet, on peut utiliser le mot clé Python `in`, exactement comme pour les listes.

```
chanteurs_disponibles = ("Jean-Jacques Goldman", "Patrick Bruel", "Renaud")
if "Michel Sardou" not in chanteurs_disponibles:
    chanteurs_disponibles = chanteurs_disponibles + ("Michel Sardou",)
```

Les p-uplets peuvent être utilisés de manière pertinente dans de nombreux cas. Tout d'abord, ils permettent de stocker plusieurs informations dans la même variable et peuvent également permettre de renvoyer plusieurs valeurs depuis une fonction. De par leur nature **immuable**, ils sont intéressants lorsque l'on souhaite manipuler des données qui ne doivent pas être modifiées par erreur. Enfin, le fait de pouvoir regrouper des éléments de différents types dans un même p-uplet permet de centraliser des données sans perdre l'avantage d'avoir plusieurs types.

```
abscisse = 5
ordonnee = 8
mon_point = (abscisse, ordonnee)

def translation(point, decalage_x, decalage_y):
    return (point[0] + decalage_x, point[1] + decalage_y)

nouveau_point = translation(mon_point, 3, 0)
```



Activité 12 : Attribution des scores

Écrire une fonction Python qui attribue le score final à une équipe à partir d'un p-uplet contenant le nom de l'équipe et les différents scores obtenus à chaque rencontre. Pour calculer le score final, effectuer une addition des scores. Par exemple :

```
score_final(("equipe 1", 3, 3, 2, 1, 2, 0))
>> ("equipe 1", 11)
```



Activité 13 : Min et Max

Écrire une fonction Python qui prend en paramètres un p-uplet et qui renvoie un tuple (p-uplet à deux éléments) contenant les éléments minimum et maximum. Par exemple :

```
min_max((87, 68, 989, 2, 67))  
>> (2, 989)
```



Activité 14 : Divorce

Écrire une fonction Python prenant en paramètres un p-uplet contenant des valeurs numériques qui renvoie un tuple de p-uplet. Le premier p-uplet contiendra les valeurs paires et le second les impaires. Par exemple :

```
separation((4, 8, 9, 6, 435))  
>> ((4, 8, 6), (9, 435))
```



Activité 15 : Exclusion

Écrire une fonction Python qui prend en paramètres deux p-uplets contenant chacun une série de valeurs et renvoyant un nouveau p-uplet contenant les éléments du premier p-uplet n'étant pas présent dans le second. Par exemple :

```
exclu = (1, 3, 7, 8, 9)  
chiffres = (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)  
difference_nuplet(chiffres, exclu)  
>> (0, 2, 4, 5, 6)
```

Les dictionnaires

Contexte 3 : Introduction

Les p-uplets et les listes sont similaires dans leur utilisation : pour accéder à un élément, il faut manipuler la structure à l'aide de l'indice de l'élément. Par exemple, pour obtenir le premier élément d'une liste (ou d'un p-uplet), on utilise l'indice 0. Il existe une structure qui diffère de cette utilisation ; c'est le dictionnaire.



Cours 5 : Dictionnaire

Un **dictionnaire** est une structure de données permettant d'associer une **valeur** à une **clef**. Les valeurs stockées sont accessibles à l'aide de leur clef, fonctionnant comme un marque page dans le dictionnaire, tout comme une définition est associée à un mot dans un dictionnaire classique. Pour définir un dictionnaire, on utilise les accolades ({}) et on définit les éléments d'un dictionnaire par : clef: valeur.

```
# deux méthodes de création d'un dictionnaire vide
dico_vide = {}
dico_vide = dict()

# création d'un dictionnaire à partir d'une liste de p-uplets de deux éléments
position_alphabet = [('a', 0), ('b', 1), ('c', 2)]
dico_alphabet = dict(position_alphabet)
print(dico_alphabet)
>> {'a': 0, 'b': 1, 'c': 2}

# création d'un dictionnaire associant à chaque fruit son prix de vente
dico_fruits = {'banane': 2.35, 'kiwi': 1.30, 'pomme': 0.75}

# ajout d'un élément dans un dictionnaire
dico_fruits['fraise'] = 5.41
```

Accès aux éléments et taille d'un dictionnaire

L'accès aux éléments, comme pour les listes et les p-uplets, s'effectue à l'aide des [], cependant au lieu d'indiquer un indice, il faut préciser la clef souhaitée : dico[clef]. Pour obtenir la taille d'un dictionnaire, il est possible d'utiliser la fonction len(). La taille d'un dictionnaire correspond au nombre de clefs présentes dans ce dictionnaire.

```
# accès au prix d'un kiwi
dico_fruits['kiwi']
>> 1.30

# taille du dictionnaire de fruits
len(dico_fruits)
>> 3
```

Modification de valeurs et appartenance

La modification d'une valeur dans un dictionnaire s'effectue de la manière suivante : dico[clef] = nouvelle_valeur. L'appartenance se vérifie à l'aide du mot-clef in comme pour les listes et les p-uplets, cependant elle ne s'applique qu'aux clefs par défaut.

```
# modification du prix d'une fraise
dico_fruits['fraise'] = 4.10
dico_fruits
>> {'banane': 2.35, 'kiwi': 1.30, 'pomme': 0.75, 'fraise': 4.10}

# test d'appartenance d'un fruit (élément clef)
'banane' in dico_fruits
>> True
```

```
# test d'appartenance d'un prix (élément valeur)
2.35 in dico_fruits
>> False
```

Fonctions disponibles sur les dictionnaires

dico.keys()	permet de récupérer toutes les clefs présentes dans le dictionnaire
dico.values()	permet de récupérer toutes les valeurs du dictionnaire
dico.items()	permet de récupérer l'ensemble des couples (clef, valeur) contenus dans le dictionnaire
dico.clear()	permet de vider un dictionnaire de son contenu
del dico[clef]	permet de supprimer un couple (clef, valeur) d'un dictionnaire grâce à la clef
dico.pop(clef)	permet de supprimer un couple (clef, valeur) d'un dictionnaire grâce à la clef et d'en récupérer la valeur



Activité 16 : Dic'Thé

Écrire une fonction Python qui prend en paramètre une liste de mots, et qui renvoie un dictionnaire tel que les clefs correspondent aux différents mots de la liste et les valeurs au nombre de fois où ces mots apparaissent. Par exemple :

```
mots = ["le", "thé", "vert", "est", "le", "meilleur", "thé", "après", "le", "thé", "noir"]
compte_mots(mots)
>> {"le" : 3, "thé" : 3, "vert" : 1, "est" : 1, "meilleur" : 1, "après" : 1, "noir" : 1}
```



Activité 17 : Ver'Dict

Écrire une fonction Python prenant en paramètre un dictionnaire dont les valeurs sont des entiers et renvoyant un p-uplet contenant la clef ou les clefs dont la valeur est la plus élevée.

```
grande_occurrence(mots)
>> ('le', 'thé')
```



Activité 18 : Tournoi des 3 sorciers

Créer un dictionnaire prenant en clef une chaîne de caractères représentant le nom d'une école et en valeur une liste de chaînes de caractères représentant les élèves inscrits dans cette école. Puis écrire une fonction Python qui pour chaque école tire un élève aléatoirement.

```
dico_ecole = {'Poudlard': ['harry', 'ron', 'hermione', 'diggory'], 'Beauxbatons': ['fleur', 'nicolas'], 'Durmstrang': ['viktor']}
selection_tournoi(dico_ecole)
>> {'Poudlard': 'diggory', 'Beauxbatons': 'fleur', 'Durmstrang': 'viktor'}
```



Activité 19 : yt yjgt hdatcctaattbcti fjt bth xcitcixdch hdci bpjkpxht

1. Écrire une fonction qui indique la position d'une lettre (minuscule) dans l'alphabet (a = 0, z = 25).

```
position_alphabet('m')
>> 12
```

2. Écrire une fonction qui fait la somme de deux entiers donnés en paramètre modulo 26.

```
decalage(12,15)
>> 1
```

3. Écrire une fonction qui prend en paramètre un mot (écrit en minuscule) et un entier, et qui décale chaque lettre du mot en fonction de l'entier en réutilisant les fonctions précédemment créées.

```
chiffrement('bonjour',15)
>> 'qdcydjg'
```

4. Écrire une fonction qui prend en paramètre un mot (écrit en minuscule) et qui renvoie un tableau contenant les mots issus des 26 possibilités de décalage, en réutilisant les fonctions précédemment créées.

```
force_brute('qdcydjg')
>> ['qdcydjg', 'redzekh', 'sfeafli', 'tgfbgmj', 'uhgchnk', 'vihdiol', 'wjiejpg', 'xkjfkqn',
'ylkglo', 'zmlhmsp', 'anmintq', 'bonjour', 'cpokpvs', 'dqplqwt', 'erqmrxu', 'fsrnsyv',
'gtsotzw', 'hutpuax', 'ivuqvby', 'jwvrwcz', 'kxwsxda', 'lyxtyeb', 'mzyuzfc', 'nazvagd',
'obawbhe', 'pcbxcif']
```

5. Créer un dictionnaire ayant pour clef les lettres de l'alphabet et contenant pour chaque lettre une liste de mots français. Puis créer une fonction qui prend en paramètre un dictionnaire et une lettre de l'alphabet et qui renvoie la liste des mots associés à cette lettre.

```
recherche(mini_dico, 'b')
>> ['bonjour', 'baignoire', 'baleine']
```

6. Écrire une fonction prenant en paramètre un tableau de mots et un dictionnaire, renvoyant une liste de p-uplet de tous les mots présents et de leur position dans le tableau également présents dans le dictionnaire.

```
supprime_mauvais_mot(['qdcydjg', 'redzekh', 'sfeafli', 'tgfbgmj', 'bonjour'], mini_dico)
>> [('bonjour', 5)]
```

7. Améliorer vos fonctions afin de pouvoir prendre en compte les espaces et pouvoir chiffrer / déchiffrer des phrases (les espaces ne seront pas chiffrés). Essayer de déchiffrer le message suivant : "lbfy de kc nomyefobd vo mrspbowoxd no mockb". Quelle valeur a été utilisée pour cacher ce message ?